

Skill Week 6

1. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
root@DUDDUKURIs  GNU nano 8.7.1  src/command_history.c *
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

int main() {
    char history[MAX_HISTORY][MAX_COMMAND];
    int count = 0;
    char command[MAX_COMMAND];

    while (1) {
        printf("history-shell> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < count; i++)
                printf("%d: %s\n", i + 1, history[i]);

            continue;
        }

        if (count < MAX_HISTORY) {
            strcpy(history[count], command);
            count++;
        } else {
            for (int i = 1; i < MAX_HISTORY; i++)
                strcpy(history[i - 1], history[i]);

            strcpy(history[MAX_HISTORY - 1], command);
        }

        printf("Command stored.\n");
    }

    return 0;
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ nano src/command_history.c
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ gcc src/command_history.c -o bin/command_history
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ ./bin/command_history
history-shell> ls
Command stored.
history-shell> pwd
Command stored.
history-shell> cd ..
Command stored.
history-shell> date
Command stored.
history-shell> whoami
Command stored.
history-shell> gcc test.c
Command stored.
history-shell> history

Command History:
1: pwd
2: cd ..
3: date
4: whoami
5: gcc test.c
history-shell> |
```

2. To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
root@DUDDUKURIs  GNU nano 8.7.1  src/pipeline_structure.c *
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

int main() {
    char input[500];
    char commands[MAX_COMMANDS][MAX_LENGTH];
    int count = 0;

    printf("Enter pipeline: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, "|");

    while (token != NULL && count < MAX_COMMANDS) {
        while (*token == ' ')
            token++;

        strcpy(commands[count], token);
        count++;

        token = strtok(NULL, "|");
    }

    printf("\nPipeline Layout:\n");

    for (int i = 0; i < count; i++)
        printf("Command %d: %s\n", i + 1, commands[i]);

    printf("\nNumber of commands: %d\n", count);
    printf("Number of pipes: %d\n", count - 1);

    return 0;
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ nano src/pipeline_structure.c
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ gcc src/pipeline_structure.c -o bin/pipeline_structure
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ ./bin/pipeline_structure
Enter pipeline: ls | grep .c | sort

Pipeline Layout:
Command 1: ls
Command 2: grep .c
Command 3: sort

Number of commands: 3
Number of pipes: 2
```

3. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
root@DUDDUKURIs  ×  +  v
GNU nano 8.7.1 src/single_pipe.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    pipe(pipefd);

    pid1 = fork();

    if (pid1 == 0) {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 == 0) {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("grep");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline completed.\n");

    return 0;
}
```

Output:

```
root@DUDDUKURIS:~/OS_Skill/Skill-Week-6$ nano src/single_pipe.c
root@DUDDUKURIS:~/OS_Skill/Skill-Week-6$ gcc src/single_pipe.c -o bin/single_pipe
root@DUDDUKURIS:~/OS_Skill/Skill-Week-6$ ./bin/single_pipe
src
Pipeline completed.
```

4.To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
root@DUDDUKURIs × + v
GNU nano 8.7.1 src/multiple_pipes.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char *commands[][4] = {
        {"ls", NULL},
        {"grep", "-c", NULL},
        {"sort", NULL}
    };

    int n = 3;
    int pipes[2][2];
    pid_t pids[3];

    for (int i = 0; i < n - 1; i++)
        pipe(pipes[i]);

    for (int i = 0; i < n; i++) {
        pids[i] = fork();

        if (pids[i] == 0) {
            if (i > 0) {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }

            if (i < n - 1) {
                dup2(pipes[i][1], STDOUT_FILENO);
            }

            for (int j = 0; j < n - 1; j++) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            execvp(commands[i][0], commands[i]);

            perror("execvp");
            exit(1);
        }
    }

    for (int i = 0; i < n - 1; i++) {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < n; i++)
        waitpid(pids[i], NULL, 0);

    printf("Long pipeline completed.\n");
    return 0;
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ nano src/multiple_pipes.c
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ gcc src/multiple_pipes.c -o bin/multiple_pipes
root@DUDDUKURIs:~/OS_Skill/Skill-Week-6$ ./bin/multiple_pipes
src
Long pipeline completed.
```